

提高作业

2.1 实践：不同目标检测算法对比

除了 YOLO，还有其他深度学习目标检测方法或模型。请选择一种不同于 YOLO 的目标检测算法或框架，完成一次可运行的目标检测 Demo，并与本讲 YOLO11n 的检测结果进行对比。

可选方向包括：

- Faster R-CNN
- SSD
- RetinaNet
- DETR / RT-DETR

要求：

- 简要介绍该方法的基本思想；
- 说明它和 YOLO 的主要区别，例如是否是 one-stage detector、速度与精度特点、适合什么场景；
- 查找该方法的官方示例或开源实现，并完成一次实际运行；
- 使用课程资料包中的至少 2 张图片进行检测，其中必须包含 soccer_practice_field.png；
- 保存运行过程截图、终端输出或检测结果图；
- 对比它和本节课 YOLO11n 的检测结果差异，重点比较：
 - 是否检测到足球 / sports ball；
 - 是否检测到人或其他目标；
 - 置信度差异；
 - 推理速度或运行体验；
 - 环境安装复杂度；
- 简要说明这个算法和 YOLO 的核心区别。

说明：本题必须完成一次实际运行。可以使用官方 Demo、开源项目、torchvision、transformers 或其他成熟工具，不要求从零实现算法。

2.2 调研：如何训练 YOLO 模型

本讲课堂只使用预训练模型做推理，不要求现场训练。请查阅 Ultralytics YOLO 相关资料，调研“如果要训练一个自己的 YOLO 足球检测模型，需要哪些步骤”。

请按流程整理，至少包含以下内容：

- **数据采集**：需要采集什么样的足球图片？应覆盖哪些场景、角度、光照、背景和距离？
- **数据标注**：YOLO 标签格式是什么？标注工具一般如何生成 .txt 标签文件？
- **数据集划分**：训练集、验证集、测试集分别有什么作用？
- **数据集配置**：YOLO 训练通常需要哪些目录和配置文件？类别名称如何写？
- **模型选择**：为什么可以先从轻量模型开始，例如 YOLO11n？
- **训练命令**：给出一个你查到的 YOLO 训练命令示例，并解释其中主要参数的含义。
- **验证与测试**：训练完成后，如何判断模型效果是否变好？
- **部署思考**：训练好的模型如何替换课堂中的 yolo11n.pt 做推理？

要求：不需要真正训练模型，但需要用自己的话把流程讲清楚。可以引用资料，但不要整段复制。

2.3 调研：CPU 推理与 GPU 推理

本讲课堂默认使用 CPU 推理，优点是安装简单、兼容性好；但在真实项目中，目标检测通常会使用 GPU 加速。

请调研并回答以下问题：

- CPU 推理和 GPU 推理的主要区别是什么？可以从速度、硬件要求、安装复杂度、功耗等角度比较。
- 为什么深度学习模型在 GPU 上通常更快？GPU 更擅长处理哪类计算？
- 如果想在 Ubuntu 中使用 NVIDIA GPU 跑 YOLO 推理，通常需要准备哪些条件？
 - NVIDIA 显卡；
 - 显卡驱动；
 - CUDA 环境；
 - 支持 GPU 的 PyTorch；
 - Ultralytics 能识别到 CUDA。

- 如何检查 PyTorch 是否能使用 GPU？请给出你查到的检查命令或 Python 代码。
- 如果能让 YOLO 使用 GPU 推理，命令或代码中通常如何指定设备？请给出示例。

要求：不要求实际实现 GPU 推理，只需要给出调研结论和思考步骤。

2.4 Aelos 机器人 Python 编程

使用 Aelos 机器人的 Python 功能，完成一个自定义功能或动作。

可选方向：

- 使用 Python 读取某个传感器数值；
- 使用 Python 控制机器人执行一个自定义动作；
- 编写一个简单的自动化流程，例如读取状态后触发动作。

提交内容：

- Python 代码或关键代码截图；
- 程序运行截图或动作执行效果照片；
- 简短说明：这个程序实现了什么功能，运行过程中遇到了什么问题。

提高作业提交要求

请将提高作业整理为 一份 PDF，包含：

- 不同目标检测算法的运行过程截图、至少 2 张检测结果图，以及与 YOLO11n 的对比分析；
- YOLO 自定义训练流程调研；
- CPU 推理与 GPU 推理对比分析；
- GPU 推理环境准备步骤；
- Aelos 机器人 Python 编程代码、效果截图与说明；
- 至少 2 条参考资料链接，并用自己的话概括其关键信息。

Answers:

2.1 实践：不同目标检测算法对比

选择 RT-DETR

- 简要介绍该方法的基本思想；

RT-DETR 沿用 DETR 系列直接预测一组目标的端到端范式，让每个真实目标只对应一个预测，完全去掉 anchor 和 NMS，并在此基础上做了三点关键改进：

(1) 高效混合编码器：将 Transformer 的尺度内特征交互与基于 CNN 的跨尺度特征融合解耦，大幅降低计算复杂度。

(2) 不确定性最小化的查询选择：RT-DETR 综合分类分数与定位置信度，选择不确定性最小的高质量特征作为解码器的初始对象查询，提升收敛速度与最终精度。

(3) 速度-精度可灵活权衡：推理时可裁剪 decoder 层数，不用重训，同一权重可适配不同算力。

- 说明它和 YOLO 的主要区别，例如是否是 one-stage detector、速度与精度特点、适合什么场景；

维度	YOLO	RT-DETR
架构范式	单阶段，CNN+anchor 密集预测	单阶段，Transformer
速度	更快	一般
精度	一般	更高
后处理	NMS 去重	无 NMS
参数量	2.6M	32M
适合场景	实时视频流、边缘设备、CPU 推理	高精度需求、GPU 充足

- 查找该方法的官方示例或开源实现，并完成一次实际运行；
- 使用课程资料包中的至少 2 张图片进行检测，其中必须包含 soccer_practice_field.png；
- 保存运行过程截图、终端输出或检测结果图；

(1) 运行 RT-DETR 脚本参见 [detect_image_rtdetr.py](#)

使用 ultralytics 所集成的 RT-DETR。

```

Detect_image_rtdetr.py
10 from ultralytics import RTDETR
11
12 # Windows + 中文路径下避免控制台编码报错
13 sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8")
14
15
16 def main():
17     new =
18     base_dir = Path(__file__).resolve().parents[1]
19     model_path = base_dir / "models" / "rtdetr-l.pt" # 首次运行: ultralytics会自动下载
20     image_paths = [
21         base_dir / "images/soccer_practice_field.png",
22         base_dir / "images/soccer_grass.png",
23         base_dir / "images/soccer_studio_closeup.png",
24         base_dir / "images/soccer_indoor_floor.png",
25         base_dir / "images/color_ball_floor_bg.png",
26         base_dir / "images/color_ball_clean.png",
27     ]
28     output_dir = base_dir / "outputs/rtdetr_predict"
29
30     model = RTDETR(str(model_path))
31
32     for image_path in image_paths:
33         results = model.predict(

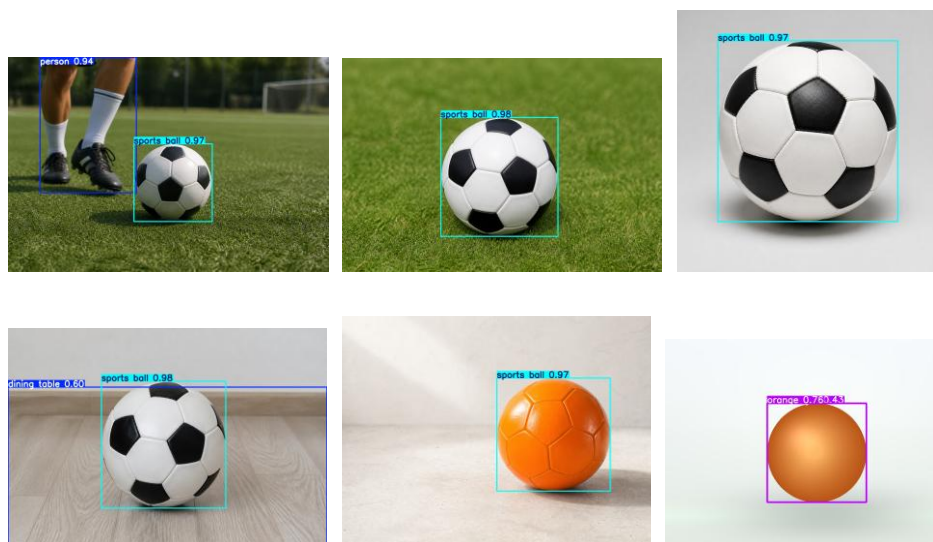
```

(2) 终端输出和检测结果图

```

Run detect_image_rtdetr
=====
image: images\soccer_practice_field.png
detections: 2
speed(ms): preprocess=7.1 inference=1650.2 postprocess=1.9
=====
[0] class_name=sports ball
confidence=0.966
bbox_xyxy=(682.8, 413.4, 977.2, 783.2)
[1] class_name=person
confidence=0.940
bbox_xyxy=(153.1, 1.2, 614.2, 646.7)
=====
visual result saved to: outputs\rtdetr_predict
=====
image: images\soccer_grass.png
detections: 1
speed(ms): preprocess=4.9 inference=714.5 postprocess=0.5
=====
[0] class_name=sports ball
confidence=0.975
bbox_xyxy=(474.8, 284.2, 1036.1, 853.4)
=====
visual result saved to: outputs\rtdetr_predict

```



- 对比它和本节课 YOLO11n 的检测结果差异，重点比较：
 - 是否检测到足球 / sports ball;
- 同 YOLO11n 相比，检测精度提升。在相同条件下，RT-DETR 能够检测出

color_ball_floor_bg、soccer_indoor_floor 、soccer_studio_closeup 中的足球。

- 是否检测到人或其他目标;

对 soccer_practice_field 图片, RT-DETR 除了检测到足球外, 仍能准确检测到球员; 对 soccer_indoor_floor 图片, 除了检测到足球外, 同时检测出餐桌。

- 置信度差异;

同 YOLO11n 相比, RT-DETR 对检测到的物体置信度有所提高。比如对 soccer_practice_field 图片, sports ball 的置信度为 0.97, 高于 YOLO11n 的 0.96; person 的置信度为 0.94, 高于 YOLO11n 的 0.91。对 soccer_grass 图片, sports ball 的置信度为 0.98, 高于 YOLO11n 的 0.44。

- 推理速度或运行体验;

同 YOLO11n 相比, RT-DETR 的推理速度慢。对 soccer_practice_field 图片, YOLO11n 的推理时长 54.8ms, 而 RT-DETR 推理时长 1361.8ms。

- 环境安装复杂度;

因为 RT-DETR 已经集成到 ultralytics, 环境安装复杂度同 YOLO11n 差别不大。

- 简要说明这个算法和 YOLO 的核心区别。

RT-DETR 同 YOLO 的核心区别在于, YOLO 是 CNN + 网格密集预测 + NMS 的实时检测范式, 优势在于速度和轻量; RT-DETR 是 Transformer + 一对一匹配的端到端范式, 优势在于精度和抗干扰, 但代价是模型更大、推理更慢。

2.2 调研：如何训练 YOLO 模型

- **数据采集：**需要采集什么样的足球图片？应覆盖哪些场景、角度、光照、背景和距离？

为了训练一个初具泛化能力的模型, 至少需要采集 2000-5000 张高质量的标注图片, 覆盖未来推理场景里所有可能的变化。

(1) 场景：室外草地、室内地板、人工草皮、水泥场、训练馆、教室地毯, 至少 3-5 类;

(2) 角度：平视 / 俯视 / 侧视 / 仰视, 不能全是某一固定机位;

(3) 光照：白天 / 阴天 / 夜晚灯光 / 强逆光 / 阴影里 / 高反光地面;

(4) 背景：纯色背景、复杂背景 (广告牌、人群、家具、彩色物体);

- (5) 距离尺寸：球在画面中占 1%到 50%不同尺寸，大小目标都要；
- (6) 遮挡：被脚挡、被人挡、被网挡，部分露出也要标注；
- (7) 外观：经典白底黑花、彩色训练球、儿童球、不同品牌；
- (8) 负样本：一部分不含球的图（含相似圆形物体，如灯、橘子），降低误检。

- **数据标注：**YOLO 标签格式是什么？标注工具一般如何生成 .txt 标签文件？

(1) YOLO 标签格式：<class_id> <x_center> <y_center> <width> <height>，class_id 代表类别编号，x_center、y_center 代表归一化的边界框中心点，width、height 代表归一化的边界框的宽度和高度。

(2) 常用标注工具 Labellmg，只需用鼠标在目标上画一个矩形框，然后从预设的类别列表中选择标签，之后就会自动保存为 YOLO 格式的.txt 标签文件。

- **数据集划分：**训练集、验证集、测试集分别有什么作用？

训练集、验证集、测试集数据互斥。

(1) 训练集 train（70-80%）：用于训练模型，梯度更新只发生在训练集。

(2) 验证集 val（10-15%）：训练过程中周期性评估，用于早停、选最优 epoch、调超参；不参与梯度更新。

(3) 测试集 test（10-15%）：训练全部结束后只评估一次，模拟“上线后”的真实表现。

- **数据集配置：**YOLO 训练通常需要哪些目录和配置文件？类别名称如何写？

参照 [ultralytics 文档](#)

- (1) YOLO 训练目录和配置文件

组织数据集：将你的数据集整理成正确的文件夹结构。你应该拥有顶级的 `images/` 和 `labels/` 目录，并在每个目录下包含 `train/` 和 `val/` 子目录。

```
dataset/
├─ images/
│   ├── train/
│   └─ val/
└─ labels/
    ├── train/
    └─ val/
```

创建 data.yaml 文件：在数据集的根目录下，创建一个 `data.yaml` 文件，用于描述数据集、类别和其他必要信息。

Name	Kind	Size
data.yaml	YAML Document	1 KB
images	Folder	--
train	Folder	--
000000000009.jpg	JPEG image	45 KB
000000000025.jpg	JPEG image	65 KB
000000000030.jpg	JPEG image	22 KB
000000000034.jpg	JPEG image	73 KB
val	Folder	--
000000000036.jpg	JPEG image	49 KB
000000000042.jpg	JPEG image	42 KB
000000000049.jpg	JPEG image	30 KB
000000000061.jpg	JPEG image	100 KB
labels	Folder	--
train	Folder	--
000000000009.txt	Plain Text	312 bytes
000000000025.txt	Plain Text	78 bytes
000000000030.txt	Plain Text	72 bytes
000000000034.txt	Plain Text	39 bytes
val	Folder	--
000000000036.txt	Plain Text	77 bytes
000000000042.txt	Plain Text	35 bytes
000000000049.txt	Plain Text	328 bytes
000000000061.txt	Plain Text	193 bytes

(2) 类别名称



- **模型选择**：为什么可以先从轻量模型开始，例如 YOLO11n？

YOLO11 有 n/s/m/l/x 五档，参数量和精度递增、速度递减。先用 n 的理由：

- (1) 参数少，训练快，能快速验证数据集的标注质量，实现快速迭代；
- (2) 所需显存资源较低；
- (3) 推理快，便于调试。

- **训练命令**：给出一个你查到的 YOLO 训练命令示例，并解释其中主要参数的含义。

YOLO 训练 CLI 命令：

```

yolo detect train model=yolo11n.pt data=football.yaml epochs=100
imgsz=640 batch=16 device=0 project=runs/football name=expl

```

主要参数（参见[训练设置](#)）：

- model=yolo11n.pt：起步权重，用 COCO 预训练的 yolo11n 做迁移学习，比从零训快很多；
- data=football.yaml：数据集配置文件；
- epochs=100：训练总轮数；
- imgsz=640：输入分辨率；
- batch=16：每批样本数；

- device=0: 单个 GPU, 无 GPU 用 cpu;
- project / name: 输出到 runs/football/exp1/

- **验证与测试:** 训练完成后, 如何判断模型效果是否变好?

训练过程中, YOLO 会在 val 集上评估并输出一系列关键性能指标, 其中最重要的包括:

- (1) mAP50: 模型在 IoU 阈值 0.5 时的平均精度, 最常用指标;
- (2) mAP50-95: 从 IoU 阈值 0.5 到 0.95 的平均 mAP, 更严格指标;
- (3) Precision 和 Recall: 衡量模型预测的准确性和其找全所有目标的能力。

基本判断:

- (1) 若 train_loss 下降、验证集 mAP 上升, 训练正常;
- (2) 若 train_loss 下降、验证集 mAP 不动甚至下降, 出现过拟合, 需要增加数据或者调整训练参数;
- (3) 若 train_loss 不降, 可能学习率太低或数据有问题。

最终判断: 用训练得到的模型在 test 集上跑检测, 观察最终的检测效果。

- **部署思考:** 训练好的模型如何替换课堂中的 yolo11n.pt 做推理?

训练完成后会得到最佳权重, 比如 best.pt, 替换课堂中使用的 yolo11n.pt 并更新 detect_image.py 脚本中的 model_path 即可进行推理。如果生产部署, 可以通过 yolo export 指令导出 onnx 等更高效的格式文件进行部署。

2.3 调研: CPU 推理与 GPU 推理

- CPU 推理和 GPU 推理的主要区别是什么? 可以从速度、硬件要求、安装复杂度、功耗等角度比较。

维度	CPU 推理	GPU 推理
速度	较慢, 实时性不足	极快
硬件要求	任何计算机均可运行, 无需特定硬件支持	需配备 NVIDIA 显卡, 并满足对应 CUDA 计算能力要求
安装复杂度	极低, 无需额外驱动和 CUDA 配置	较高, 需安装显卡驱动、CUDA Toolkit, 并确保 PyTorch 版本与 CUDA 版本兼容匹配
依赖体积	CPU 版 torch 约 200 MB	GPU 版 torch+cuXX 约 2GB
推理功耗	整机 30-125W	单卡 100-450W, 但每瓦算力远高于 CPU
适用场景	适合教学/边缘设备	适合训练/视频流实时检测/大批量推理

- 为什么深度学习模型在 GPU 上通常更快？GPU 更擅长处理哪类计算？

深度学习的核心计算为大规模矩阵乘法、卷积、张量运算，高度可并行。GPU 相对于 CPU 可以拥有成千上万轻量核心（比如 RTX 4090，拥有 16384 个 CUDA 核 + 512 个 Tensor 核），使得同一指令可以并行作用在大量数据上，加快深度学习的核心计算。

GPU 更擅长处理稠密矩阵乘法、卷积运算、并行运算等。

- 如果想在 Ubuntu 中使用 NVIDIA GPU 跑 YOLO 推理，通常需要准备哪些条件？

- NVIDIA 显卡；

必需硬件基础（Pascal 架构及以上，如 GTX 10 系、RTX 20/30/40 系列、A100/V100 等）

- 显卡驱动；

安装 NVIDIA 官方闭源驱动。安装完成后运行 `nvidia-smi` 验证驱动是否正常加载

- CUDA 环境；

CUDA Toolkit 是 GPU 编程的核心运行时库，需确保版本与显卡驱动兼容

- 支持 GPU 的 PyTorch；

需安装包含 CUDA 支持的 PyTorch 版本，而非 CPU-only 版本

- Ultralytics 能识别到 CUDA。

安装 ultralytics 包后，框架会自动检测 PyTorch 的 CUDA 能力，若检测成功则默认使用 GPU。通过运行 `yolo checks`，可以看到显卡型号。

- 如何检查 PyTorch 是否能使用 GPU？请给出你查到的检查命令或 Python 代码。

一行命令：

```
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.device_count(), torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'no gpu')"
```

- 如果要想 YOLO 使用 GPU 推理，命令或代码中通常如何指定设备？请给出示例。

命令行 (加 device=0, 表示第一张 GPU):

```
yolo predict model=models/yolo11n.pt  
source=images/soccer_practice_field.png conf=0.25 device=0
```

多卡用 device=0,1 或 device=cuda:0; 强制用 CPU, 则设置 device=cpu。

2.4 Aelos 机器人 Python 编程

- (1) python 代码参见 [aelos_demo.py](#)
- (2) 参见[程序运行截图](#)、[动作执行效果截图](#)
- (3) 参见[简短说明](#)

2.5 参考资料

[1] Ultralytics 官方文档: <https://docs.ultralytics.com/>

YOLO 系列模型的官方使用文档, 含 train/predict/val/export 全部 CLI 与 Python 接口; 同时已集成 RT-DETR、SAM 等新模型, 本次 RT-DETR 加载就是直接 from ultralytics import RTDETR。

[2] Ultralytics 训练教程: <https://docs.ultralytics.com/modes/train/>

YOLO 训练的标准流程、参数表、断点续训、自动 batch size 估计。本次 2.2 训练命令示例 (yolo detect train ...) 参考此页面。

[3] COCO 数据集类别表: <https://cocodataset.org/>

YOLO11n 预训练用的 80 类目录, 其中 sports ball 代表球类的统一类别 (id=32), 无 football 类, 这是本次足球被识别为 sports ball 的根因。